

Forbidden Island FEI Edition

Table of Contents

1. Avances 26 de septiembre de 2025.....	1
1.1. Casos de uso	1
1.2. Porcentajes de contribución	1
1.3. Forbbiden.Server	1
1.4. Iniciar sesión como servicio	1
1.5. Registrar usuario como servicio	5
1.6. Editar perfil de usuario como servicio	9
1.7. Menú principal (Nuevas funciones).....	11
1.8. Nuevo juego (Nuevas funciones)	13
1.9. Forbbiden.Test.....	14
1.10. Uso de IA.....	17
2. Casos de Prueba	18
3. Objetivos de la próxima iteración	20
3.1. Diseño	20
3.2. Implementación de casos de uso	20
3.3. Deuda técnica	20

Chapter 1. Avances 26 de septiembre de 2025

1.1. Casos de uso

- Forbbiden.Server
- Iniciar sesión como servicio
- Registrar usuario como servicio
- Editar perfil de usuario como servicio
- Menú principal (nuevas funciones)
- Nuevo Juego (nuevas funciones)
- Forbbiden.Test

1.2. Porcentajes de contribución

- Leonardo 50%
- Gabriel 50%

1.3. Forbbiden.Server

1.4. Iniciar sesión como servicio

Responsables:

- Gabriel: Lógica con el servidor
- Lenoardo: Actualización de GUI

Objetivos para la próxima iteración: Mejorar la interfaz de usuario para que se vea más bonita

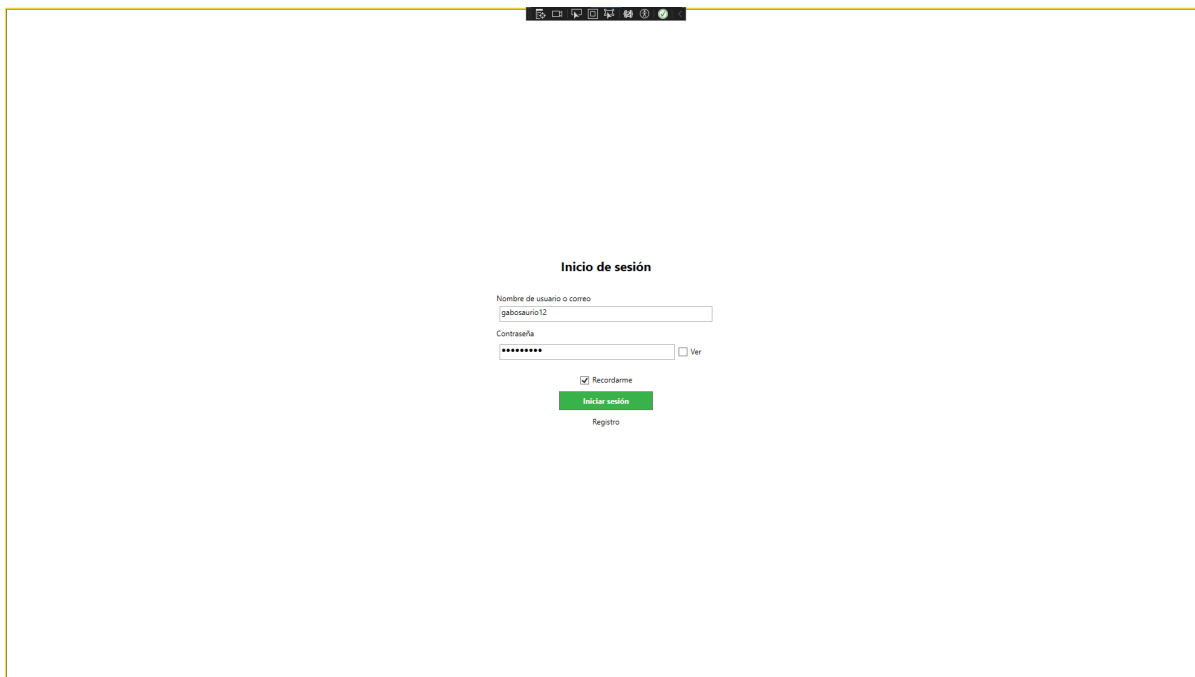


Figura 1. Inicio de sesión en español

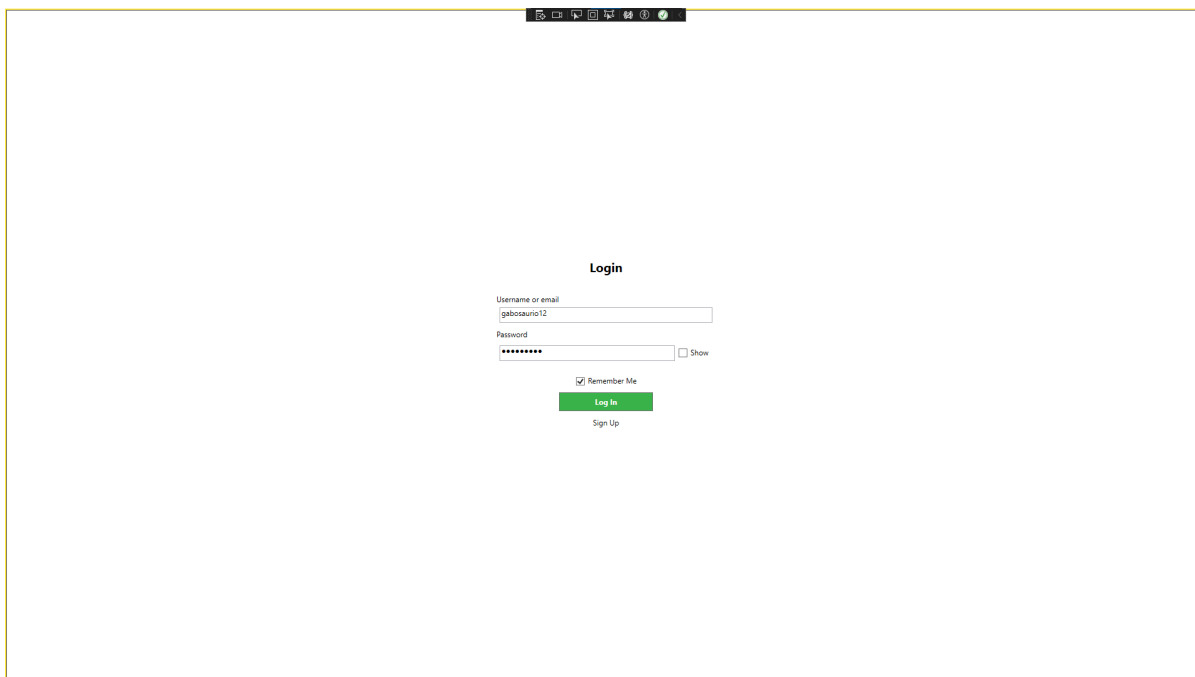


Figura 2. Inicio de sesión en inglés

1.4.1. Código del cliente

```
public partial class LoginPage : Page
{
    private static readonly ILog log = LogManager.GetLogger(typeof(LoginPage));

    private void BtnLogin_Click(object sender, RoutedEventArgs e)
    {
        ResetFieldColors();

        var client = new ProfileManagerClient();
```

```

Player searchPlayer = client.GetPlayerByUsername(txtBUsername.Text);

if (searchPlayer == null)
{
    MessageBox.Show("El nombre de usuario no existe.");
    txtBkUser.Foreground = Brushes.Red;
}
else
{
    string password = "";
    if (chkPassword.IsChecked == true)
    {
        password = txtBPasswordVisible.Text;
    }
    else
    {
        password = pwdBPassword.Password;
    }

    if (BCrypt.Net.BCrypt.Verify(password, searchPlayer.PlayerPassword))
    {
        if (client.Login(searchPlayer))
        {
            Properties.Settings.Default.rememberLogin =
chkRememberMe.IsChecked == true;
            Properties.Settings.Default.Save();

            var logWindow = Window.GetWindow(this) as LogWindow;
            logWindow?.Close();
            log.Info($"Usuario '{searchPlayer.PlayerUsername}' logged in.");
        }
        else
        {
            log.Warn($"Login failed for user
'{searchPlayer.PlayerUsername}'");
            MessageBox.Show("Error al iniciar sesión. Por favor, inténtelo de
nuevo.");
        }
    }
    else
    {
        txtBkPassword.Foreground = Brushes.Red;
        MessageBox.Show("Contraseña incorrecta");
    }
}
}
}

```

1.4.2. Código del servidor

```
[ServiceBehavior(InstanceContextMode = InstanceContextMode.Single)]
public class ProfileManager : IProfileManager
{
    private static readonly ILogger log = LogManager.GetLogger(typeof(ProfileManager));

    public bool Login(Contracts.Player player)
    {
        bool success = true;
        using (var db = new Forbbiden_FEIEntities())
        {
            db.LoginPlayer.Add(new LoginPlayer
            {
                login_player_id = player.PlayerId,
            });
            try
            {
                db.SaveChanges();
            }
            catch (DbEntityValidationException ex)
            {
                success = false;
                log.Error("ProfileManager.cs", ex);
            }
            catch (DbUpdateException ex)
            {
                success = false;
                log.Error("ProfileManager.cs", ex);
            }
            catch (Exception ex)
            {
                success = false;
                log.Error("ProfileManager.cs", ex);
            }
        }

        return success;
    }

    public Contracts.Player GetPlayerByUsername(string username)
    {
        using (var db = new Forbbiden_FEIEntities())
        {
            var playerResult = db.Player.FirstOrDefault(u => u.player_username ==
username);
            if (playerResult != null)
            {
                return new Contracts.Player
                {

```

```

        PlayerId = playerResult.player_id,
        PlayerUsername = playerResult.player_username,
        PlayerPassword = playerResult.player_password,
        PlayerEmail = playerResult.player_email
    };
}
return null;
}
}
}

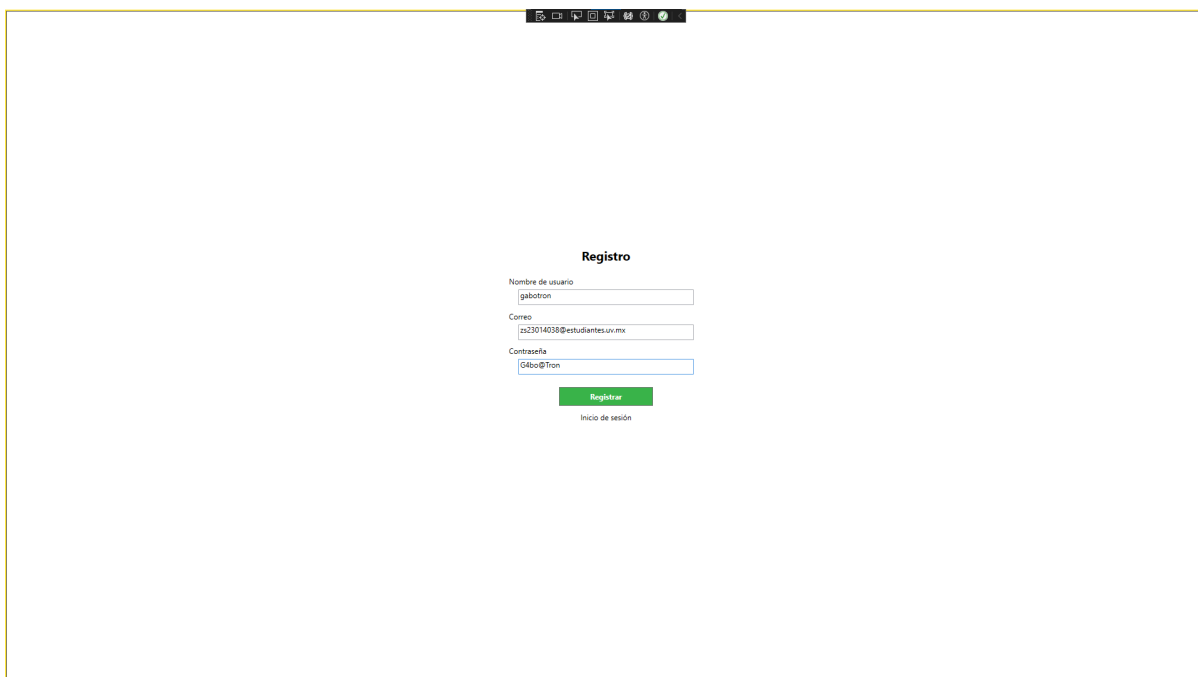
```

1.5. Registrar usuario como servicio

Responsables:

- Gabriel: Lógica con el servidor
- Lenoardo: Actualización de GUI

Objetivos para la próxima iteración: Mejorar la interfaz de usuario para que se vea más bonita



The screenshot shows a web browser window with a registration form. The form is titled "Registro" and contains three input fields: "Nombre de usuario" (Username) with the value "gabotron", "Correo" (Email) with the value "zs23014038@estudiantes.uv.mx", and "Contraseña" (Password) with the value "G4bo@Tron". Below the input fields is a green "Registrar" button and a link for "Inicio de sesión" (Login).

Figura 3. Registro de usuario en español

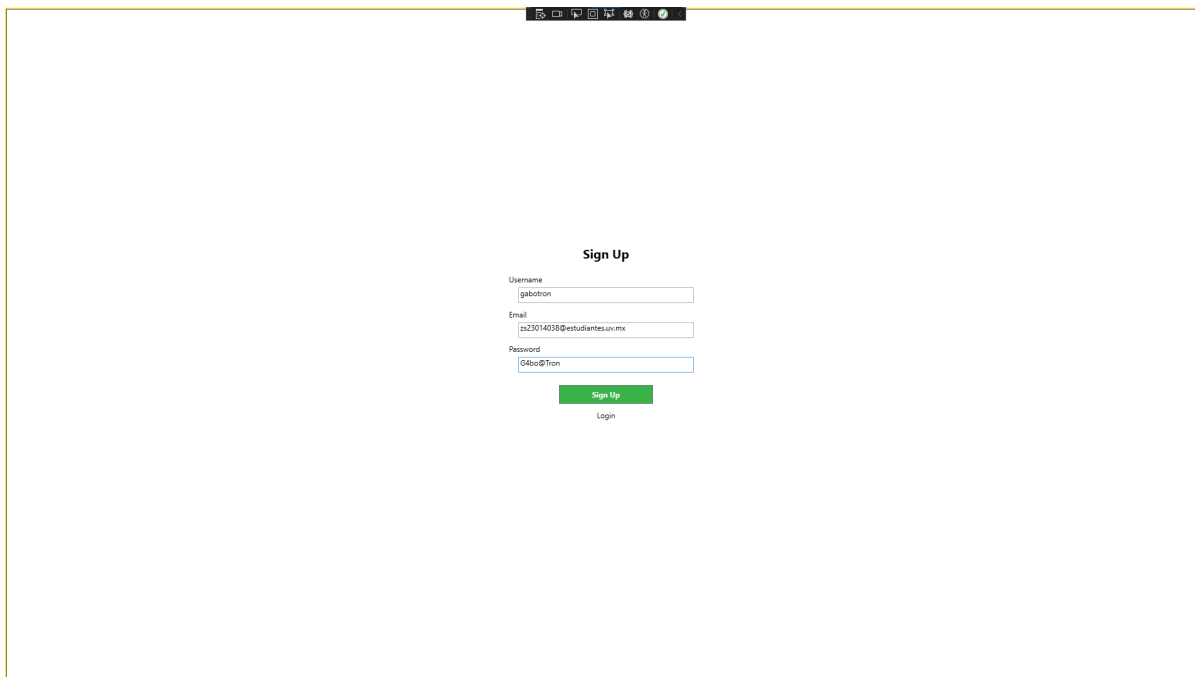


Figura 4. Registro de usuario en inglés

1.5.1. Código del cliente

```
public partial class SignupPage : Page
{
    private static readonly ILog log = LogManager.GetLogger(typeof(SignupPage));

    private bool ValidatePlayer(ref Player player, ProfileManagerClient client)
    {
        bool isValid = true;

        if (!ValidatePassword(player.PlayerPassword))
        {
            MessageBox.Show("La contraseña debe tener al menos 8 caracteres, una mayúscula, una minúscula, un número y un carácter especial.");
            isValid = false;
        }
        if (string.IsNullOrEmpty(player.PlayerUsername))
        {
            MessageBox.Show("El nombre de usuario no puede estar vacío.");
            isValid = false;
        }
        if (player.PlayerUsername.Contains(" "))
        {
            MessageBox.Show("El nombre de usuario no puede contener espacios.");
            isValid = false;
        }
        if (!client.IsUsernameAvailable(player.PlayerUsername))
        {
            MessageBox.Show("El nombre de usuario ya está en uso.");
            isValid = false;
        }
    }
}
```



```

        if (!client.ValidateEmail(player.PlayerEmail))
        {
            MessageBox.Show("El correo electrónico es inválido o ya está en uso.");
            isValid = false;
        }

        return isValid;
    }

    private void SignupButton_Click(object sender, RoutedEventArgs e)
    {
        var client = new ProfileManagerClient();

        var player = new Player
        {
            PlayerUsername = txtBUsername.Text,
            PlayerEmail = txtBEmail.Text,
            PlayerPassword = txtBPassword.Text
        };

        if (ValidatePlayer(ref player, client))
        {
            player.PlayerPassword =
BCrypt.Net.BCrypt.HashPassword(player.PlayerPassword);
            if (client.SignUp(player))
            {
                log.Info($"Usuario {player.PlayerUsername} sent.");
                MessageBox.Show("Usuario registrado exitosamente.");
                NavigationService.Navigate(new LoginPage());
            }
            else
            {
                MessageBox.Show("Error al registrar el usuario. Inténtelo de nuevo más
tarde.");
            }
        }
    }
}

```

1.5.2. Código del servidor

```

[ServiceBehavior(InstanceContextMode = InstanceContextMode.Single)]
public class ProfileManager : IProfileManager
{
    private static readonly ILog log = LogManager.GetLogger(typeof(ProfileManager));

    public bool ValidateEmail(string email)
    {
        if (string.IsNullOrEmpty(email))

```

```

        {
            return false;
        }
        if (email.Contains("@"))
        {
            using (var db = new Forbbiden_FEIEntities())
            {
                var emailResult = db.Player.FirstOrDefault(p => p.player_email ==
email);
                return emailResult == null;
            }
        }

        return false;
    }

    public bool IsUsernameAvailable(string username)
    {
        using (var db = new Forbbiden_FEIEntities())
        {
            try
            {
                var playerResult = db.Player.FirstOrDefault(u => u.player_username ==
username);
                return playerResult == null;
            }
            catch (Exception ex)
            {
                log.Error("ProfileManager.cs", ex);
                return false;
            }
        }
    }

    public bool SendEmail(string email)
    {
        bool success = true;
        string receiver = email;
        string emisor = "forbbidenislandfei@gmail.com";
        MailMessage message = new MailMessage(emisor, receiver);
        message.Subject = "Register confirmation";
        message.Body = "Your account has been succesfully created, welcome to
Forbbiden Island FEI Edition. Enjoy the adventure!";
        SmtpClient client = new SmtpClient("smtp.gmail.com");
        client.Port = 587;
        string emailCode = Properties.Settings.Default.emailCode;
        client.Credentials = new System.Net.NetworkCredential(emisor, emailCode);
        client.EnableSsl = true;

        try
        {

```

```

        client.Send(message);
    }
    catch (Exception ex)
    {
        log.Error("ProfileManager.cs", ex);
        success = false;
    }

    return success;
}

public bool SignUp(Contracts.Player player)
{
    bool success = true;
    using (var db = new Forbbiden_FEIEntities())
    {
        Player newPlayer = new Player
        {
            player_username = player.PlayerUsername,
            player_password = player.PlayerPassword,
            player_email = player.PlayerEmail
        };
        try
        {
            db.Player.Add(newPlayer);
            db.SaveChanges();
            SendEmail(newPlayer.player_email);
        }
        catch (DbEntityValidationException ex)
        {
            log.Error("ProfileManager.cs", ex);
            success = false;
        }
        catch (DbUpdateException ex)
        {
            log.Error("ProfileManager.cs", ex);
            success = false;
        }
    }
    return success;
}
}

```

1.6. Editar perfil de usuario como servicio

Responsables:

- Gabriel: Lógica y GUI

<

Profile



gabosaurio12

Import an image from your device to use as an avatar

Upload New Avatar

Username

gabosaurio12

Name

Gabriel Antonio

Email

mazinger.gl@gmail.com



gabosaurio



gabox



gabosaurio35



Gabriel

Save Changes

Discard Changes

Figura 5. Página de perfil en español

<

Profile



gabosaurio12

Import an image from your device to use as an avatar

Upload New Avatar

Username

gabosaurio12

Name

Gabriel Antonio

Email

mazinger.gl@gmail.com



gabosaurio



gabox



gabosaurio35



Gabriel

Save Changes

Discard Changes

Figura 6. Página de perfil en inglés

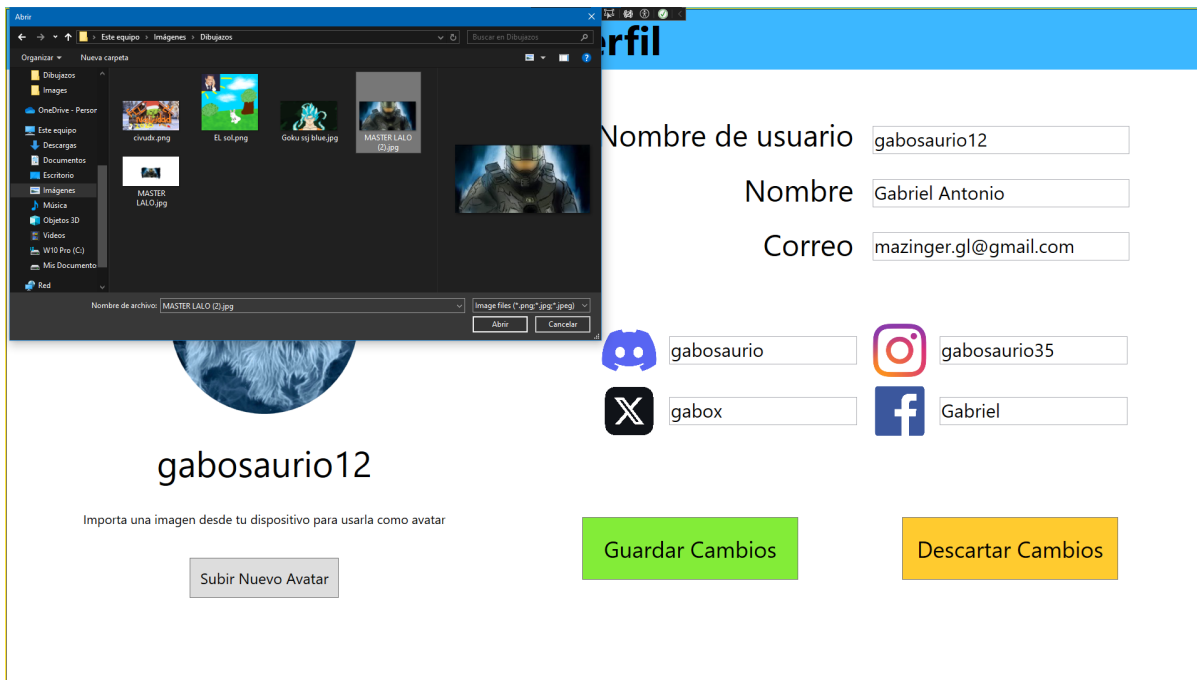


Figura 7. Subir avatar en español

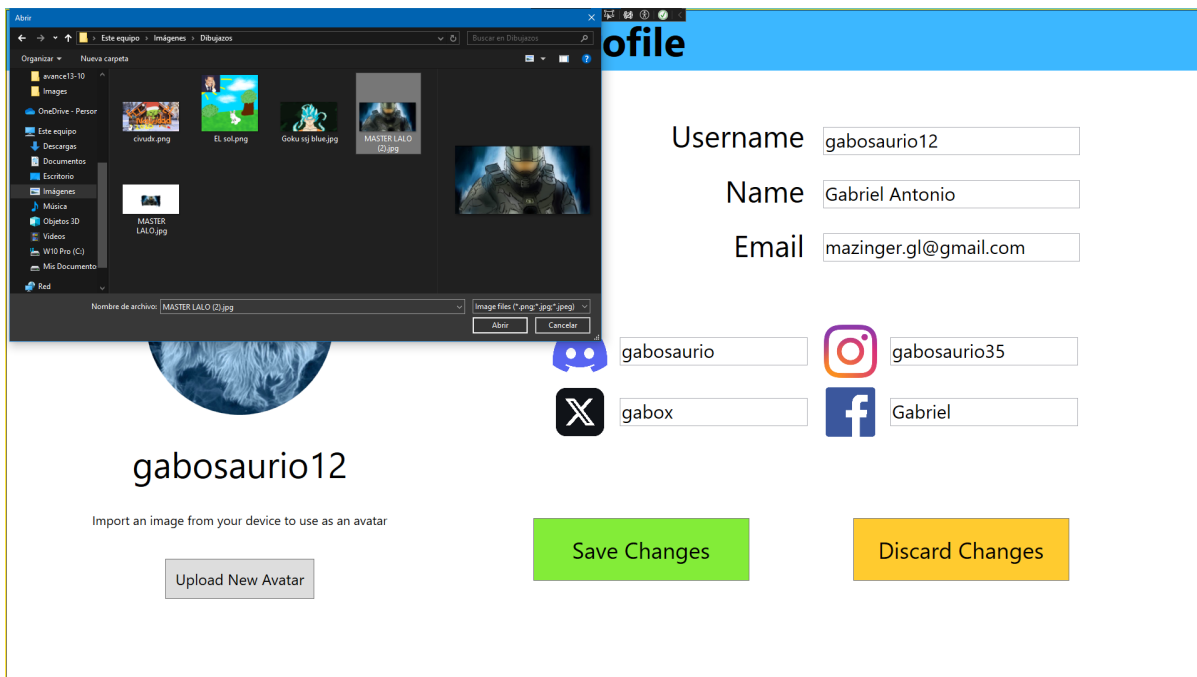


Figura 8. Subir avatar en inglés

1.7. Menú principal (Nuevas funciones)

Responsables:

- Leonardo: Lógica y GUI



Figura 9. Menú principal en español



Figura 10. Menú principal en inglés



Figura 11. Menú principal loggeado en español

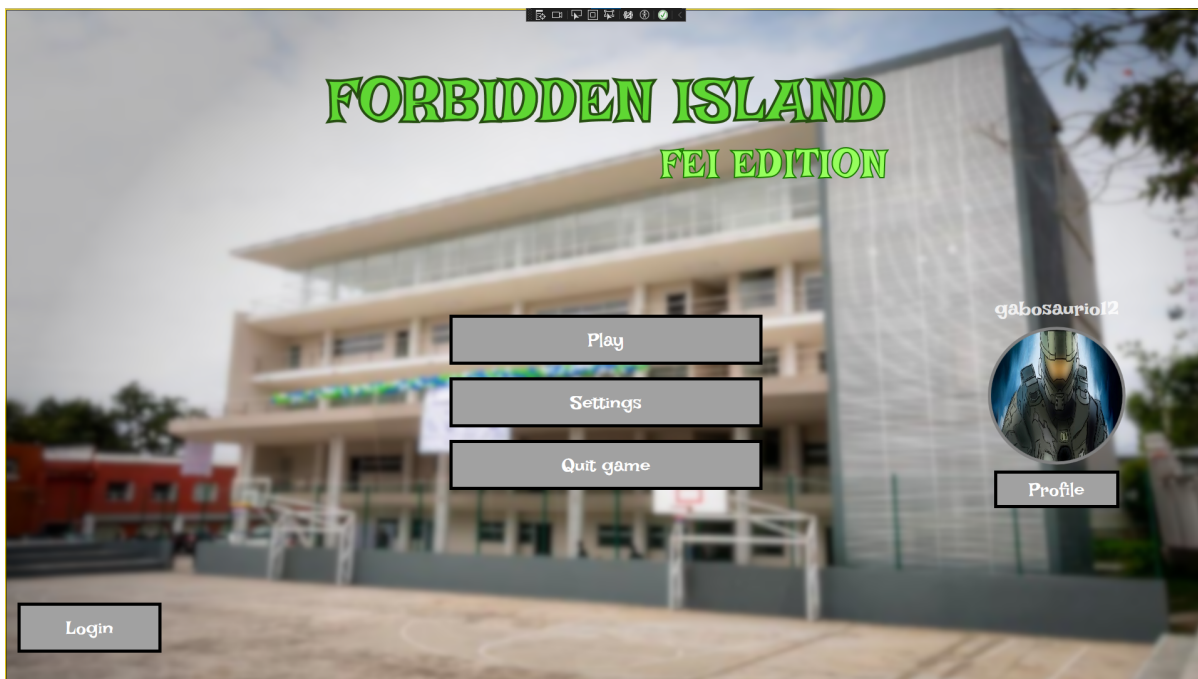


Figura 12. Menú pirncipal loggeado en inglés

1.8. Nuevo juego (Nuevas funciones)

Reponsables:

- Leonardo: Lógica y GUI



Figura 13. Nuevo juego en español

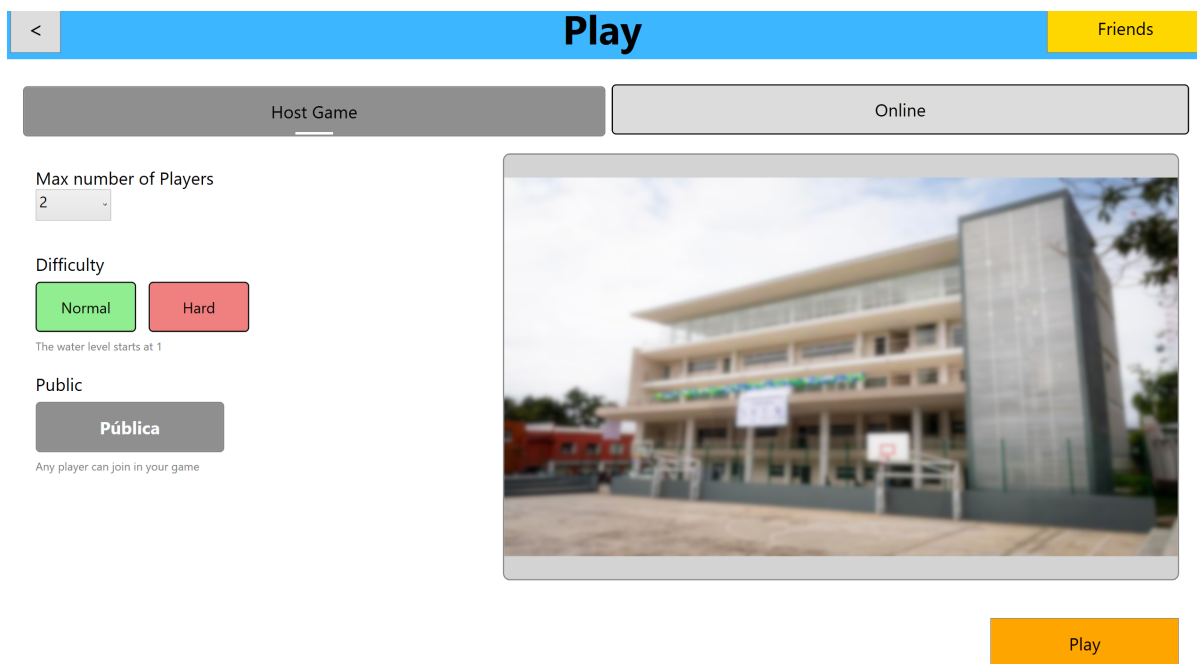


Figura 14. Nuevo juego en inglés

1.9. Forbbiden.Test

Responsables:

- Gabriel: Diseño e implementación

1.9.1. TestProfileManager

Para más detalle sobre las pruebas ver el reporte de casos de prueba.

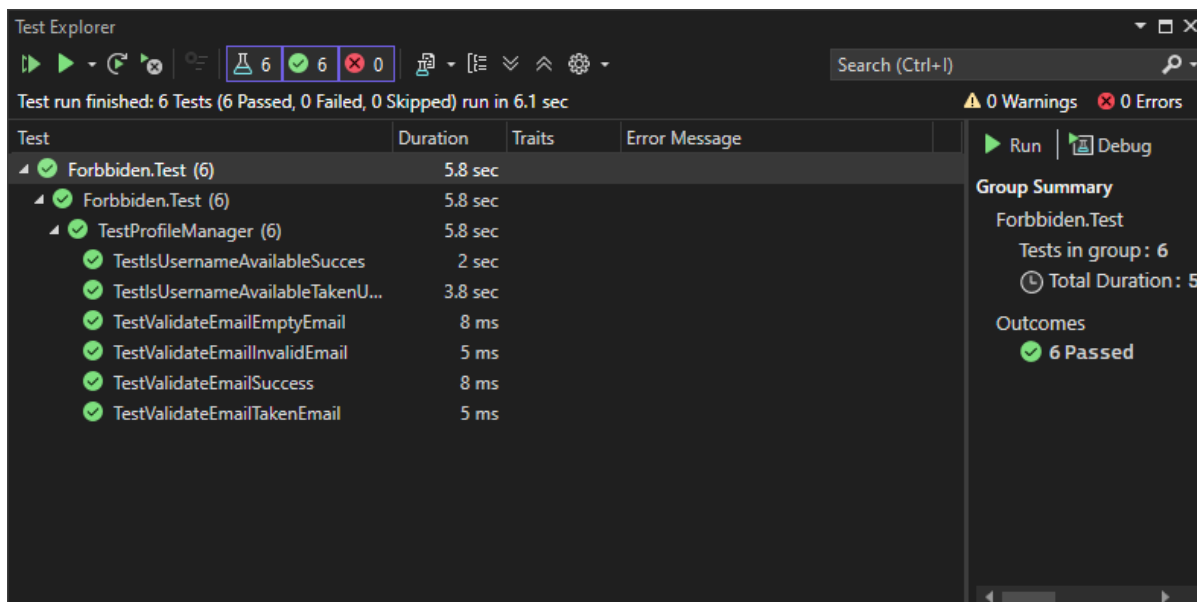


Figura 15. Pruebas

Código de pruebas

```
[TestFixture]
public class TestProfileManager
{
    [OneTimeSetUp]
    public void Setup()
    {
        var client = new ProfileManagerClient();

        Player player = new Player
        {
            PlayerUsername = "testUser",
            PlayerPassword = "T3st_pass",
            PlayerEmail = "testuser@email.net"
        };

        client.SignUpAsync(player);
    }

    [OneTimeTearDown]
    public void TearDown()
    {
        var client = new ProfileManagerClient();
        string username = "testUser";
        client.DeletePlayerByUsernameAsync(username);
    }

    [Test]
    public async Task TestValidateEmailSuccess()
    {
        var client = new ProfileManagerClient();
```

```

        string email = "randomEmail@email.net";

        var result = await client.ValidateEmailAsync(email);
        Assert.That(result, Is.True, "result should be true");
    }

    [Test]
    public async Task TestValidateEmailInvalidEmail()
    {
        var client = new ProfileManagerClient();
        string email = "randomEmail.com";

        var result = await client.ValidateEmailAsync(email);
        Assert.That(result, Is.False, "result should be false");
    }

    [Test]
    public async Task TestValidateEmailEmptyEmail()
    {
        var client = new ProfileManagerClient();
        string email = "";

        var result = await client.ValidateEmailAsync(email);
        Assert.That(result, Is.False, "result should be false");
    }

    [Test]
    public async Task TestValidateEmailTakenEmail()
    {
        var client = new ProfileManagerClient();
        string email = "testuser@email.net";

        var result = await client.ValidateEmailAsync(email);
        Assert.That(result, Is.False, "result should be false");
    }

    [Test]
    public async Task TestIsUsernameAvailableSucces()
    {
        var client = new ProfileManagerClient();
        string username = "testUser098";

        var result = await client.IsUsernameAvailableAsync(username);
        Assert.That(result, Is.True, "result should be true");
    }

    [Test]
    public async Task TestIsUsernameAvailableTakenUsername()
    {
        var client = new ProfileManagerClient();
        string username = "testUser";
    }

```

```
        var result = await client.IsUsernameAvailableAsync(username);
        Assert.That(result, Is.False, "result should be false");
    }
}
```

1.10. Uso de IA

1.10.1. Gabriel

- Uso de ChatGPT para entender mejor qué hacía cada elemento o control de app.xaml, app.config y cómo configurarlos para que Forbidden.Server pudiera correr.
- Uso de ChatGPT para entender el modelo Cliente-Servidor.
- Uso de ChatGPT para comprender cómo maneja Entity Framework los datos y las consultas
- Uso de ChatGPT para aprender a usar Visual Studio 2022 y su particular forma de refactorizar (como renombrar archivos o namespaces)
- Uso de ChatGPT para consultar los métodos de los elementos de WPF (como los de Ellipse, CheckBox, TextBox, etc)
 - Aquí incluyo el solicitar una Cheat-Sheet de los generales de los elementos (le pedí que los comparara con los de JavaFX para entender un poco mejor)

Nota: No uso la IA para que resuelva los problemas que surgen, la uso para entenderlos y yo poder encontrar una solución, así que generalmente le pregunto cosas como "Qué hace este fragmento del App.config" o "Leí sobre esta forma de hacer consultas con EF, cuál es la diferencia con esta otra" o cosas por el estilo. Me gusta leer primero documentación o soluciones de algún usuario en foros y ya después si me agrada alguna pero no la entiendo al 100% le pido a ChatGPT que me explique lo que no entiendo, esto con la intención de usarlo como una herramienta y no como una solución.

1.10.2. Leonardo

- Uso de ChatGPT para entender qué herramientas se suelen utilizar para implementar ciertas funcionalidades o abordar aspectos de UI.
- Uso de ChatGPT para aprender cómo traducir prototipos visuales a código XAML en WPF.
- Uso de ChatGPT para la generación de estilos de la interfaz
 - Para diseñar y personalizar los estilos de los botones.
 - Para poder conseguir la apariencia prevista durante los prototipos.
 - Para investigar cómo se suelen organizar los temas visuales de la aplicación.
 - Para explorar buenas prácticas de UI y consistencia en la apariencia.

Chapter 2. Casos de Prueba

ID	Nombre	Objetivo	Prioridad	Precondiciones	Entradas	Resultados Esperados	Resultados Observados	Resultado
PMT01	TestValidateEmailSuccess	Comprobar si el email ingresado tiene un formato válido y no está registrado en la base de datos	Alta	PRE01: Haber ingresado un email	<i>randomEmail@email.net</i>	<i>True</i>	Regresa <i>true</i>	Pasó
PMT02	TestValidateEmailInvalidEmail	Comprobar que no se acepten emails sin un @	Alta	PRE01: Haber ingresado un email	<i>randomEmail.com</i>	<i>False</i>	Regresa <i>false</i>	Pasó
PMT03	TestValidateEmailEmptyEmail	Comprobar que no se acepte un email vacío	Alta	PRE01: Haber ingresado un email	""	<i>False</i>	Regresa <i>false</i>	Pasó
PMT04	TestValidateEmailTakenEmail	Comprobar que no se acepte un email que ya esté registrado	Alta	PRE01: Haber ingresado un email PRE02: Tener el email <i>testuser@email.net</i> registrado	<i>testuser@email.net</i>	<i>False</i>	Regresa <i>false</i>	Pasó

PMT05	TestIsUsernameAvailableSuccess	Comprobar que el username no esté registrado en la base de datos	Alta	PRE01: Haber ingresado un username	<i>testUser098</i>	<i>True</i>	Regresa <i>true</i>	Pasó
PMT06	TestIsUsernameAvailableTakenUsername	Comprobar que no se acepte un username registrado en la base de datos	Alta	PRE01: Haber ingresado un username	<i>testUser</i>	<i>False</i>	Regresa <i>true</i>	Pasó

Chapter 3. Objetivos de la próxima iteración

Se planea abarcar los siguientes puntos en la próxima iteración.

3.1. Diseño

- Descripciones de caso de uso principales (más o menos 5)
- Diagramas de clases (más o menos 3 casos de uso)
- Diagramas de secuencia (más o menos 3 casos de uso)

3.2. Implementación de casos de uso

- Lista de amigos
- Chat entre amigos
- Lobby
- Decorar GUI Login y Registro
- Terminar GUI Nueva Partida

3.3. Deuda técnica

- Mejorar el manejo de excepciones
- Internacionalizar todos los pop ups
- Estandarizar comentarios (al idioma inglés)